

Buổi 2

Kiến trúc REST và HTTP Fundamentals

Sáu ràng buộc kiến trúc REST · HTTP methods & status codes · Headers · Content negotiation · Bài tập Flask



tách biệt UI khỏi data store

mọi thứ là resource

stateless · cacheable · layered

Mục tiêu bài học

Liệt kê sáu nguyên tắc REST và HTTP methods

Phân tích vì sao REST phù hợp với kiến trúc Web hiện đại.

Thiết kế HTTP requests và responses cơ bản

Viết đúng method, status code, header cho từng tình huống thực tế.

Đánh giá một API có tuân thủ REST hay không

Nhận diện vi phạm từ ví dụ thực — SOAP-wrapper, RPC-trong-URL, verb-noun.

| Nội dung chính

Nền tảng khái niệm

REST là gì, lịch sử, so sánh với SOAP/RPC, vì sao gắn với Web.

HTTP chi tiết

Anatomy, methods, idempotency, status codes, headers, content negotiation.

Sáu ràng buộc REST

Client-Server, Stateless, Cacheable, Uniform Interface, Layered, Code on Demand.

Thực hành Flask

3 bài tập xây /books RESTful, có chạy thử & bắt lỗi.

| Vì sao REST phù hợp với kiến trúc Web?

01

Stateless từng hop

Mỗi proxy/CDN xử lý độc lập vì không có session, mỗi hop chỉ cần nhìn message.

02

Cache

Browser cache, CDN, server cache, phải dùng cùng cơ chế để CDN hoạt động.

03

Uniform contract

Mọi server dùng cùng phương thức định danh (URI), client không cần biết trước.

04

Layered deployment

Cache, filter, gateway là lớp trung gian, client và server không biết nhau.

Năm 2000, Web đã có **~17 triệu site**, phục vụ **~300 triệu người dùng**.

HTTP chịu tải rất cao nhưng không có cách chuẩn hóa cách thiết kế API.

17M

Website năm 2000 (Netcraft)

300M

Người dùng Web cuối 2000

1.0

HTTP/1.1 — persistent conn
RFC 2616, 1999

9

Ràng buộc REST (sáu chính + ba phụ)
Fielding 2000, §5

Sáu ràng buộc kiến trúc REST

I Client–Server

BẮT BUỘC

Tách **giao diện người dùng** khỏi **lưu trữ dữ liệu**. Mỗi bên phát triển độc lập.

II Stateless

BẮT BUỘC

Mỗi request chứa **tất cả** thông tin cần để xử lý — server không nhớ state giữa hai request.

III Cacheable

BẮT BUỘC

Response có thể đánh dấu **cacheable** hoặc **not**; client có thể tái sử dụng.

IV Uniform Interface

BẮT BUỘC · SÂU NHẤT

Một **hợp đồng chung** giữa client và server — có 4 ràng buộc con (xem slide 12).

V Layered System

BẮT BUỘC

Client không biết nó đang nói chuyện với server cuối hay lớp trung gian — kiến trúc phân lớp.

VI Code on Demand

TÙY CHỌN

Server có thể gửi **code chạy được** (JavaScript, Applet) — ràng buộc duy nhất có thể bỏ qua.

Nhầm lẫn phổ biến

Gọi “đây là REST API” rồi đặt session trong cookie hay trả về HTML thay vì JSON.

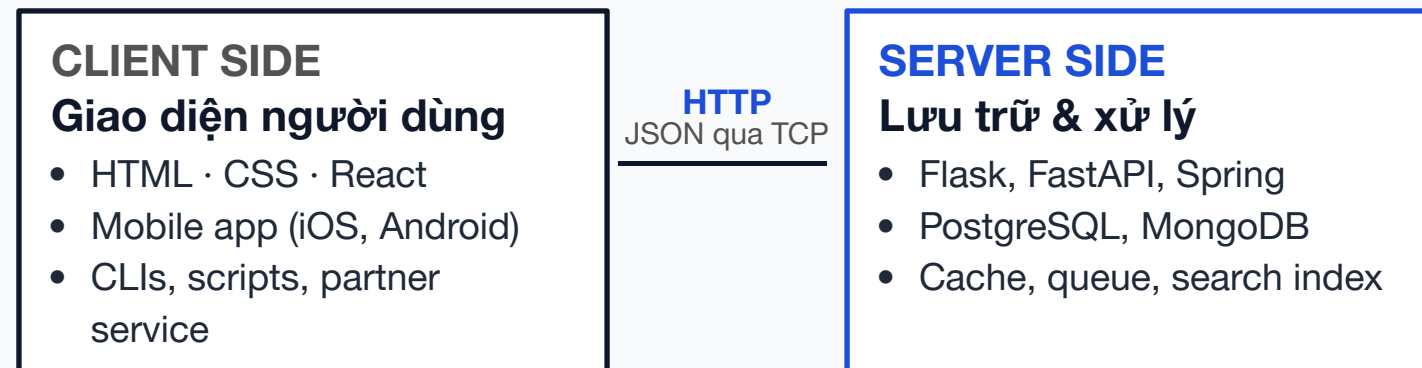
REST không phù hợp khi

Streaming realtime, file transfer gigabyte, peer-to-peer

Tách giao diện khỏi lưu trữ

SƠ ĐỒ

Hai bên, một hợp đồng



- ✓ Client biết URI; server biết cách xử lý: hai bên phát triển độc lập.
- ✓ Server có thể đổi DB mà client không nhận ra.
- ✓ Frontend có thể nhúng iframe gọi trực tiếp server backend.

PHÁT BIỂU GỐC

“Tách các **concern** giữa giao diện người dùng và lưu trữ dữ liệu — cải thiện tính di động (portability) của giao diện và khả năng mở rộng (scalability) của server.”

— Fielding 2000

... nghe có vẻ hiển nhiên?

Trước REST (1990s), nhiều hệ thống **trộn UI và data**: server gửi HTML render sẵn với session ID nhúng trong form ẩn. Việc thay đổi giao diện = viết lại từ đầu.

Client-server yêu cầu client **không biết** server lưu trữ thế nào và ngược lại, chỉ trao đổi qua định dạng chuẩn (URI + representation).

Mỗi request phải tự nó đầy đủ

*“Mỗi request từ client **chứa tất cả thông tin** cần thiết: server **không lưu session state** giữa các request.”*

— Fielding 2000, §5.1.5

KHÔNG STATELESS · SAI

POST /orders

Cookie: sid=ABC123

→ server tra bảng session

→ tìm user_id, cart_id

STATELESS · ĐÚNG

POST /orders

Authorization: Bearer eyJ...

```
{  
  "cart_id": "c_91f3",  
  "items": [...]  
}
```

VÌ SAO QUAN TRỌNG

Scale ngang

Mọi instance đều xử lý được request, không sticky session; load balancer round-robin thoải mái.

Quan sát được

Mỗi request là event độc lập: log, audit, debug đều local; log chứa đủ context.

Tái sử dụng cache

Cache ở gateway serve lại response cũ nếu URL và điều kiện giống: stateless mới cache hiệu quả.

RÀNG BUỘC KHÔNG CÓ NGHĨA LÀ “KHÔNG BAO GIỜ LƯU TRỮ”

Stateless nói về **request**: phải đầy đủ thông tin. Server vẫn lưu DB, cache, audit log, chúng chỉ KHÔNG ảnh hưởng đến cách hiểu request hiện tại.

Đánh dấu response: có thể cache hay không

Mỗi response tự khai báo: **cacheable** hay **not cacheable**.

Khai báo này cho phép client, proxy, CDN lưu và tái sử dụng → giảm tải server, latency giảm.

FLASK · CACHE

```
# books_api.py — Flask cache 5 phút
from flask import Flask, jsonify, make_response
app = Flask(__name__)

@app.get("/books/<int:bid>")
def get_book(bid):
    book = db.find(bid)
    if not book:
        return {"error": "not found"}, 404

    resp = make_response(jsonify(book))
    # cache 5 phút (client + CDN)
    resp.headers["Cache-Control"] = "public, max-age=300"
    return resp
```

CACHE ĐƯỢC

Cache-Control: public, max-age=300

Cache-Control: private, max-age=60

KHÔNG CACHE ĐƯỢC

Cache-Control: no-store

Cache-Control: no-cache, no-store, must-revalidate

VALIDATION

ETag: "d41d8cd98f00b204..."

Last-Modified: Sat, 15 Aug 2026 10:30 GMT

TẠI SAO REST BẮT BUỘC

HTTP caching đã có sẵn, REST yêu cầu *dùng* nó, không tự phát minh.

CACHE HAY KHÔNG CACHE?

GET /books/42 an toàn cache — dữ liệu hiếm đổi. **GET /users/me/orders** phải **private, no-store** — dữ liệu cá nhân.

Bốn ràng buộc con của Uniform Interface

Đây là phần *phân biệt REST* với các kiểu mạng khác (RPC, network-style). Mọi thành phần giao tiếp qua cùng một hợp đồng.

A · IDENTIFICATION OF RESOURCES

Mỗi resource có một URI ổn định

```
GET https://api.bookstore.io/books/42
POST https://api.bookstore.io/orders
DELETE https://api.bookstore.io/users/me/sessions/sess_8f3
```

URI chỉ *định danh*, **không** phải action. So sánh với RPC: `/getBook?id=42` lẫn lộn động từ và danh từ.

B · MANIPULATION THROUGH REPRESENTATIONS

Thao tác qua biểu diễn (representation)

Client không sửa resource trực tiếp, gửi **biểu diễn** mới (JSON, XML); server tự quyết định biểu diễn nội bộ (bảng, document, graph).

```
PATCH /books/42
{"price": 29.99}
// server hiểu và tự cập nhật field giá
```

C · SELF-DESCRIPTIVE MESSAGES

Mỗi message tự mô tả nó

Content-Type, **Accept**, status code phải đủ để một client chưa biết gì vẫn có thể hiểu message đó là gì.

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /orders/o_91f3
```

D · HYPERMEDIA AS THE ENGINE OF APPLICATION STATE (HATEOAS)

Response chứa liên kết tới bước tiếp theo

Client khám phá API qua response — server là nguồn duy nhất của state chuyển tiếp. Trình bày chi tiết ở slide 15.

REST trong công nghiệp thường chỉ thỏa 3/4 ràng buộc. HATEOAS thường bị bỏ qua.

Vẫn có thể là một API tốt, nhưng nên gọi là “RESTful-ish” hoặc “resource-oriented” để tránh nhầm lẫn với REST đầy đủ.

Resource là gì, URI viết sao cho đúng?

Resource = bất kỳ thứ gì có thể đặt tên, có thể thay đổi trạng thái theo thời gian.
URI là *nhãn* để trỏ tới resource, không phải bản thân resource.

URI ĐÚNG CÚ PHÁP

https://	api.bookstore.io	/v2/	books/	42	/reviews
scheme — https	authority	URI versioning	resource — books	identifier — 42	sub-resource

NÊN · ĐÚNG NGHĨA RESOURCE

GET /books
GET /books/42
GET /books/42/reviews
POST /orders
PUT /users/me
PATCH /orders/o_91f3
DELETE /sessions/sess_a8d

✓ Danh từ, số nhiều, phân cấp rõ ràng.

KHÔNG NÊN · RPC HÓA URL

GET /getBook?id=42
POST /createOrder
GET /listAllUsers
POST /updateBookInfo
GET /query?table=book&order=price

✗ Động từ trong URL, tham số hành động → đây là RPC nhúng HTTP

1. Danh từ số nhiều, không động từ

2. Phân cấp tự nhiên

3. Không để đuôi file (.json)

4. Lowercase, kebab-case

Resource ≠ biểu diễn của nó

Cùng một resource có thể được biểu diễn bằng **nhiều định dạng**: JSON cho mobile, XML cho partner cũ, HTML cho admin web.
URI vẫn chỉ một → chỉ payload thay đổi.

RESOURCE (trừu tượng)

Cuốn sách tên “Clean Code”

Có id nội bộ, metadata (tiêu đề, tác giả, ISBN), giá. Tồn tại độc lập với cách gửi đi.



REPRESENTATIONS (cụ thể)

application/json

application/xml

text/csv

text/html

```
{
  "id": 42,
  "title": "Clean Code",
  "price": 29.99,
  "currency": "USD"
}
```

Hai kiểu representation

Resource representation: biểu diễn hiện trạng resource.

Metadata representation: thông tin kèm theo (Last-Modified, ETag) cho phép cache, validation, concurrency.

Lựa chọn representation bằng cách nào?

GET /books/42 **HTTP/1.1**

Accept: application/json

Accept-Language: vi-VN, en;q=0.8

Header **Accept** cho phép client *đề nghị* định dạng.
Đây là cơ chế chính của **content negotiation**

Hầu hết API hiện đại chỉ trả **JSON** → đơn giản

Một API “mở” hỗ trợ JSON/XML/CSV qua Accept + 406 khi không thoả mãn → dễ tích hợp partner.

HATEOAS — Hypermedia điều khiển state ứng dụng

Response không chỉ là dữ liệu, nó còn cho biết **có thể làm gì tiếp theo** thông qua liên kết.
Client *không đọc tài liệu* để biết URL, mà đọc chính response để khám phá.

KHÔNG HATEOAS

Client hard-code URL hành động

```
// Dev đọc tài liệu để biết URL cancel
const order = await fetch('/orders/91');
const r = await fetch(
  `/orders/${order.id}/cancel`,
  {method: 'POST'});
! Server đổi URL → client vỡ
```

CÓ HATEOAS

Client đọc link từ response

```
// Server trả kèm liên kết
{
  "id": "o_91",
  "status": "pending",
  "_links": {
    "self": {"href": "/orders/o_91"},
    "cancel": {"href": "/orders/o_91/cancel",
    "method": "POST"}
  }
}
```

FLASK · CHÈN LINK ĐỘNG VÀO RESPONSE

```
@app.get("/orders/<oid>")
def get_order(oid):
    order = Order.query.get_or_404(oid)
    return jsonify({
        "id": order.id,
        "_links": {
            "cancel": {"href": f"/orders/{oid}/cancel",
            "method": "POST"}
        }
    })
```

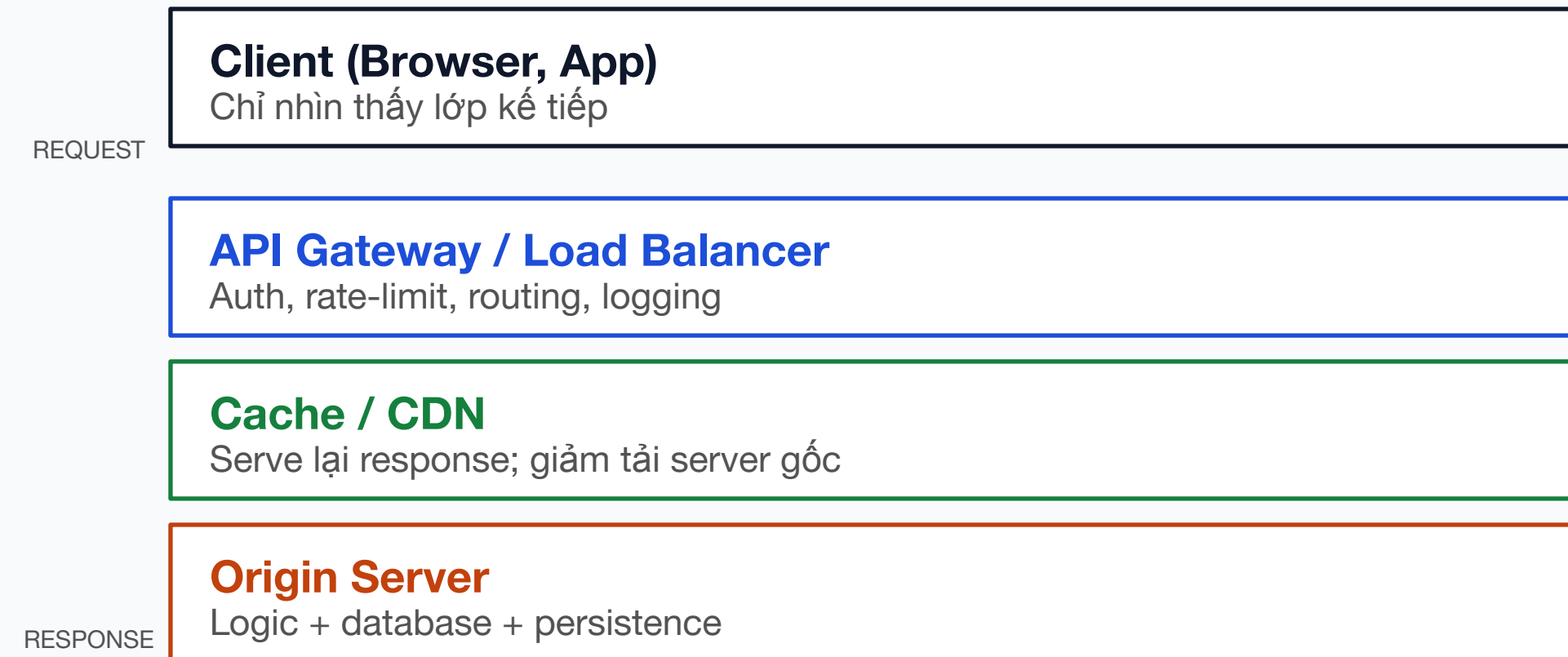
Trong thực tế

- GitHub, Stripe, Shopify có *một dạng* hypermedia nhưng không theo HATEOAS đầy đủ.
- Buổi 11 sẽ học chi tiết **HAL, JSON:API, Siren** — các chuẩn hypermedia

Phổ biến trong production: *chỉ chèn link quan trọng* (next, cancel, edit)
— không cần đầy đủ mọi thao tác.

Hệ thống được phân lớp

SƠ ĐỒ 4 LỚP



TẠI SAO CẦN PHÂN LỚP?

- Mỗi lớp chỉ thấy lớp kề — thay đổi bên trong không ảnh hưởng các lớp khác.
- CDN lỗi → có thể bypass sang origin; client không cần biết.
- Tách **scalability concern** — mỗi lớp scale theo cách riêng.

CODE ON DEMAND (TÙY CHỌN)

Server có thể gửi **code chạy được trên client**, thường là JavaScript (1990s: Java applets). REST không bắt buộc, và khi gửi script thì khả năng cache và tính quan sát (visibility) *giảm*.

Trong thực tế: backend REST thường chỉ trả JSON; chỉ web truyền thống mới dùng Code on Demand. Đây là lý do nó là ràng buộc duy nhất được phép bỏ qua.

CASE STUDY

Khi Shopee Việt Nam tách **API Gateway** ra khỏi backend chính (2020), họ phục vụ 2 tỷ request/ngày, gateway làm rate-limit + auth, origin chỉ lo business logic. Nếu gateway quá tải, họ tăng gateway instances; origin server không cần khởi động thêm.

Bốn hiểu lầm thường gặp về REST

“REST = API trả về JSON qua HTTP”

Sai — REST định nghĩa kiểu kiến trúc, không phải định dạng. JSON chỉ là một representation.

“POST cập nhật một phần, PUT tạo mới”

Ngược — PUT tạo mới hoặc thay thế trạng thái của tài nguyên tại URI đã biết; POST *tạo mới*. PATCH mới cập nhật một phần.

“RESTful API có cache thì không cần Authorization”

Sai — cache theo URL và Authorization scope. Dữ liệu user-A không cache chung user-B.

“Cứ có 6 ràng buộc là REST, thiếu 1 là RPC”

Sai — RPC vs REST *không* nhị phân; có phổ (spectrum). Thực tế fielding ủng hộ pair-wise thay vì severity.

Khi thiết kế API, tự hỏi:

- API có tuân thủ các ràng buộc bắt buộc của REST không?
- Uniform Interface có thực sự được bảo đảm không?
- Nếu có chủ đích vi phạm một ràng buộc, lợi ích và đánh đổi là gì?

Request line, status line, headers, body

REQUEST

POST /api/v2/orders?urgent=true **HTTP/1.1** ← request-line: method + target + version

Host: api.bookstore.io ← required từ HTTP/1.1

Accept: application/json

Authorization: Bearer eyJhbGciOiJIUzI1Ni...

Content-Type: application/json

Content-Length: 87 ← dòng trống phân tách headers / body

{"cart_id": "c_91", "items": [...]}

RESPONSE

HTTP/1.1 201 Created ← status-line: version + code + reason

Content-Type: application/json; charset=utf-8

Location: /orders/o_91f3

Cache-Control: no-store

ETag: "5d4a-91f3a2c4e8b80"

Content-Length: 142 ← dòng trống phân tách headers / body

{"id": "o_91f3", "status": "pending", "total": 248.50}

CẤU TRÚC · MỘT HTTP MESSAGE

Request = method + target + version + headers + (body) · **Response** = version + status-code + reason-phrase + headers + (body) · cách nhau bằng CRLF CRLF.

Các method thường dùng

METHOD	VAI TRÒ	IDEMPOTENT	SAFE	VÍ DỤ
GET	Đọc một hoặc nhiều resource	✓ Có	✓ Có	GET /books/42
POST	Tạo resource mới hoặc xử lý generic	✗ Không	✗ Không	POST /orders {...}
PUT	Thay thế toàn bộ resource	✓ Có	✗ Không	PUT /users/me {...toàn bộ...}
PATCH	Cập nhật một phần (RFC 5789)	— Không*	✗ Không	PATCH /books/42 {"price":29}
DELETE	Xóa resource	✓ Có	✗ Không	DELETE /books/42
HEAD	Như GET nhưng chỉ trả headers	✓ Có	✓ Có	HEAD /books/42 (check tồn tại)
OPTIONS	Mô tả khả năng của resource	✓ Có	✓ Có	OPTIONS /books → Allow: GET,POST
CONNECT	Mở TCP tunnel (proxy HTTPS)	—	—	CONNECT api.example.com:443

Lưu ý

POST ≠ “sửa”

Thực tế nhiều API dùng POST cho cả update, vì PUT *thay thế toàn bộ*, dễ sai sót.

PUT ≠ “tạo mới”

PUT *thay thế* resource; nếu chưa tồn tại có thể tạo, nhưng RFC gợi ý dùng POST cho create.

PATCH cần JSON Patch?

Không bắt buộc — PATCH thường dùng JSON Merge Patch (RFC 7396) hoặc JSON Patch (RFC 6902).

Idempotency & Safety: vì sao quan trọng?

IDEMPOTENT · RFC 9110 §9.2.2

Gọi N lần, kết quả giống

Request hiện tại và tương lai **đều cho cùng một kết quả** về phía server
→ không tạo thêm state mới. Áp dụng: GET, PUT, DELETE, HEAD, OPTIONS.

DELETE /users/me/sessions/abc

Gọi 1 lần: 204 No Content

Gọi 100 lần: 204 No Content (đã xóa)

SAFE · RFC 9110 §9.2.1

Không thay đổi server state

Method **chỉ đọc** → không tạo, sửa, xóa; có thể cache được. Áp dụng: GET, HEAD, OPTIONS.

GET /users/me/orders

Đọc toàn bộ đơn hàng của tôi

Server **KHÔNG** log lại “đã đọc”

VÌ SAO QUAN TRỌNG?

Retry an toàn

Client mất kết nối giữa chừng, cần gửi lại: với idempotent method, gửi lại N lần cho cùng một kết quả.

Cache hoạt động đúng

Browser/CDN chỉ cache được những method safe. Nếu GET secretly ghi log, cache sẽ bị sai.

Predictable behavior

Người dùng nhấn F5 / pre-fetch / crawler — chỉ GET mới an toàn để họ thực thi.

POST CÓ IDEMPOTENCY-KEY · GIẢI PHÁP RETRY AN TOÀN

POST /payments Idempotency-Key: ord_91f3x_attempt2 {...}

Server lưu response theo key trong 24h — lần retry lấy lại kết quả cũ thay vì charge hai lần.

Thành công và chuyển tiếp

2xx · THÀNH CÔNG

200

OK

Trả resource kèm body.
GET, PUT, PATCH thành công?

201

Created

Tạo resource mới. **Nên** có
Location trả URI mới.

202

Accepted

Server đã *nhận* nhưng chưa xử
lý xong: job queue, async.

204

No Content

Xử lý xong nhưng **không có**
body. DELETE thường trả 204.

206

Partial Content

Range request: trả một phần
file lớn; ít gặp trong JSON
API.

3xx · CHUYỂN TIẾP

301

Moved Permanently

Resource chuyển vĩnh viễn —
client cache URL mới. *Không*
nên dùng cho POST.

304

Not Modified

Cache còn hiệu lực — client
dùng bản lưu. Tương tác với
If-None-Match.

307

Temporary Redirect

Tạm thời. **Giữ nguyên method**
— thích hợp cho POST.

308

Permanent Redirect

Vĩnh viễn + giữ method.
Versioning: URI cũ → URI mới.

302

Found — tránh dùng

RFC 9110: hành vi *không rõ*
ràng. Dùng 307 thay.

FLASK · 201 Created với Location header

```
@app.post("/books")
def create_book():
    book = Book.create(**request.json)
    resp = make_response(
        jsonify(book.to_dict()), 201
    )
    resp.headers["Location"] = f"/books/{book.id}"
    return resp
```

RESPONSE

HTTP/1.1 **201 Created**

Location: </books/42>

→ client có thể [GET /books/42](#) để xem chi tiết

Lưu ý: Location phải tuyệt đối (absolute URL) nếu client ở domain khác.

Lỗi client & lỗi server

4xx · LỖI TỪ CLIENT

400 Bad Request

Request malformed: JSON sai cú pháp, thiếu field bắt buộc. Client **phải sửa** trước khi gửi lại.

403 Forbidden

Đã xác thực nhưng **không đủ quyền**.
401 = thiếu auth; 403 = có auth nhưng bị từ chối.

409 Conflict

State xung đột với request. Hai user cùng update một record.

401 Unauthorized

Chưa xác thực. Thiếu/sai token. RFC 9110 định nghĩa là “unauthenticated”.

404 Not Found

Resource không tồn tại — hoặc server *giả vờ* không tồn tại để ẩn thông tin.

422 Unprocessable

JSON hợp lệ nhưng **vi phạm business rule** (email sai format, age < 0).

5xx · LỖI TỪ SERVER

500 Internal Error

Lỗi không kỳ vọng. *Không* lộ stack trace cho client.

503 Service Unavailable

Tạm thời không phục vụ — deploying, quá tải. **Phải** gửi Retry-After.

429 Too Many Requests

Rate limit vượt ngưỡng. Kèm Retry-After. RFC 6585.

502 Bad Gateway

Gateway nhận response không hợp lệ từ upstream. Hay gặp nginx+flask/gunicorn.

504 Gateway Timeout

Gateway chờ upstream quá lâu. 503 = từ chối; 504 = chờ quá thời gian.

501 Not Implemented

Server không hỗ trợ method này. OPTIONS thường bị treo nếu chưa implement.

FLASK · ERROR HANDLER

```
@app.errorhandler(404)
def not_found(e): return jsonify(error="not_found"), 404

@app.errorhandler(422)
def validation(e): errors = [err.msg for err in e.errors]; return jsonify(errors=errors), 422

# Đăng ký handler FLAT cho JSON — không trả HTML mặc định của Flask
# 500 nên log stack trace rồi trả "internal_error" generic
```

Năm lớp Status codes

Mỗi status code là **số 3 chữ số**, chữ số đầu tiên cho biết *lớp phản hồi*. Hai chữ số sau do IETF định nghĩa: không cần biết hết, nhưng phải nhớ **các nhóm chính**.

1xx Informational	2xx Success	3xx Redirection	4xx Client Error	5xx Server Error
MỤC ĐÍCH Response tạm thời: request đang tiếp tục xử lý. PHỔ BIẾN 100 Continue 101 Switching Protocols 103 Early Hints API JSON phần lớn <i>không cần dùng</i> trực tiếp — trừ server-streaming.	MỤC ĐÍCH Request thành công: client nhận được resource/result. PHỔ BIẾN 200 OK 201 Created 202 Accepted 204 No Content 206 Partial Content <i>Nhìn vào code, không vào body</i> — 2xx là nhóm duy nhất chắc chắn OK.	MỤC ĐÍCH Client cần thực hiện thêm hành động (thường gọi lại URL khác). PHỔ BIẾN 301 Moved Permanently 302 Found (temporary) 304 Not Modified 307 Temporary Redirect 308 Permanent Redirect 301/308 thay đổi method sau redirect — tránh dùng cho POST (sửa 307/308).	MỤC ĐÍCH Lỗi từ phía client: sai cú pháp, không hợp lệ, không có quyền. PHỔ BIẾN 400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found 409 Conflict 422 Unprocessable 429 Too Many Requests	MỤC ĐÍCH Lỗi từ phía server, không phải lỗi client. PHỔ BIẾN 500 Internal 501 Not Implemented 502 Bad Gateway 503 Unavailable 504 Gateway Timeout Request phải <i>idempotent</i> để retry an toàn.

Tóm lại:
2xx thành công · **3xx** chuyển tiếp · **4xx** client phải sửa · **5xx** server nên retry · **1xx** thông tin tạm thời.

Các headers hay gặp

CONTENT · THÔNG TIN PAYLOAD

Content-Type

MIME của body: application/json

Content-Length

Byte của body. Flask tự set, không nên hard-code.

Content-Encoding

Nén: gzip, br, deflate.

Accept

Client yêu cầu MIME *đề nghị*

Accept-Language

Ngôn ngữ ưu tiên: vi-VN, en;q=0.8

CACHE · KIỂM SOÁT ĐỆM

Cache-Control

Header chính kiểm soát: max-age, no-store, public, private.

ETag

Định danh duy nhất của version. Client gửi If-None-Match để validate.

Last-Modified

Thời điểm sửa cuối. Client dùng If-Modified-Since.

Vary

Cache *phụ thuộc* header nào: Vary: Accept-Language

Age

Response đã nằm trong cache bao nhiêu giây.

AUTH & MISC · XÁC THỰC + KHÁC

Authorization

Bearer JWT: Authorization: Bearer eyJhbGciOi....

WWW-Authenticate

Challenge gửi kèm 401.

Location

URI mới (201 Created, 3xx redirect).

Retry-After

Kèm 429 và 503 — bao nhiêu giây nữa thử lại.

User-Agent

Client tự nhận diện. *Không* dùng để xác thực.

FLASK QUICK-VIEW

```
resp.headers["Cache-Control"] = "public, max-age=300"
resp.headers["ETag"] = hashlib.md5(body).hexdigest()
request.headers.get("Authorization")
```


Client đề nghị, server chọn

cùng một URI có thể trả về nhiều representation. Client *đề nghị* định dạng, server *chọn* (hoặc trả 406 nếu không thoả mãn).

KỊCH BẢN 1: CLIENT MOBILE — SERVER TRẢ JSON

① Client gửi

```
GET /books/42 HTTP/1.1
Host: api.bookstore.io
Accept: application/json
Accept-Language: vi-VN
```



② Server trả về

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Language: vi
{"id":42, "title":"Clean Code"}
```

KỊCH BẢN 2: 406 NOT ACCEPTABLE

```
GET /books/42 HTTP/1.1
Accept: application/pdf
```

HTTP/1.1 406 Not Acceptable

```
Content-Type: application/json
{"error":"only JSON", "supported":["application/json"]}
```

Q-VALUE · ĐỘ ƯU TIÊN

Client xếp hạng nhiều lựa chọn bằng q-value (0...1, mặc định 1).

```
Accept: text/html,
application/json;q=0.9,
application/xml;q=0.8
```

Ưu tiên *HTML*, nếu không có mới tính JSON, sau cùng XML.

FLASK: `fmt = request.accept_mimetypes.best_match(["application/json", "text/csv"])`

Xây **/books** — GET list + POST create

Yêu cầu tối thiểu: danh sách rỗng ban đầu, có **POST** thêm sách, **GET** đọc một sách. Tạo file `app.py`; chạy `flask --app app run --debug`.

```
# app.py — bài 1: GET /books, POST /books
from flask import Flask, jsonify, request
app = Flask(__name__)
BOOKS = []
_next_id = 1
# — GET /books — trả danh sách
@app.get("/books")
deflist_books():
    return jsonify({
        "data": BOOKS,
        "total": len(BOOKS)
    }), 200
# — POST /books — tạo mới
@app.post("/books")
defcreate_book():
    global _next_id
    ifnot request.is_json:
        return jsonify(error="expected JSON"), 415
    p = request.get_json(silent=True) or {}
    t = (p.get("title") or "").strip()
    a = (p.get("author") or "").strip()
    ifnot t ornot a:
        return jsonify(error="title and author required"), 422
    book = {"id": _next_id, "title": t, "author": a}
    BOOKS.append(book); _next_id += 1
    resp = make_response(jsonify(book), 201)
    resp.headers["Location"] = f"/books/{book['id']}"
    return resp
```

CHẠY THỬ BẰNG CURL

```
# list khi rỗng
$ curl localhost:5000/books

# tạo cuốn
$ curl -X POST \
localhost:5000/books \
-H "Content-Type: application/json" \
-d '{"title":"Clean Code",
"author":"R. Martin"}
```

STATUS CODE MONG ĐỢI

```
POST OK ..... 201
POST thiếu field ... 422
POST sai JSON ..... 400
POST thiếu Content-Type .. 415
GET danh sách rỗng ..... 200
{"data":[],"total":0}
```

Tip: thử POST thiếu title xem 422 — Flask chỉ trả HTML; muốn JSON thì jsonify error.

PUT thay thế · PATCH một phần · DELETE xóa

Lưu ý: **PUT thay toàn bộ**, PATCH giữ field không gửi, DELETE *idempotent*.

```
# — GET /books/<id> — cache 60s
@app.get("/books/<int:bid>")
def fetch(bid):
    i = next((k for k,b in enumerate(BOOKS) if b["id"]==bid), None)
    if i is None: return jsonify(error="not found"), 404
    resp = make_response(jsonify(BOOKS[i]), 200)
    resp.headers["Cache-Control"]="max-age=60"; return resp
# — PUT — thay toàn bộ, title+author bắt buộc
@app.put("/books/<int:bid>")
def put(bid):
    i = next((k for k,b in enumerate(BOOKS) if b["id"]==bid), None)
    if i is None: return jsonify(error="not found"), 404
    p = request.get_json(silent=True) or {}
    t,a = p.get("title"), p.get("author")
    if not t or not a: return jsonify(error="need title+author"), 422
    BOOKS[i]={"id":bid,"title":t.strip(),"author":a.strip(),
    "isbn":p.get("isbn"),"price":p.get("price")}
    return jsonify(BOOKS[i]), 200
# — PATCH — chỉ cập nhật field có trong body
@app.patch("/books/<int:bid>")
def patch(bid):
    i = next((k for k,b in enumerate(BOOKS) if b["id"]==bid), None)
    if i is None: return jsonify(error="not found"), 404
    p = request.get_json(silent=True) or {}
    if p.get("price", 0) < 0:
        return jsonify(error="price must be positive"), 422
    for k in "title author isbn price".split():
        if k in p: BOOKS[i][k] = p[k]
    return jsonify(BOOKS[i]), 200
# — DELETE — idempotent, trả 204
@app.delete("/books/<int:bid>")
def delete(bid):
    i = next((k for k,b in enumerate(BOOKS)
    if b["id"]==bid), None)
    if i is None: return jsonify(error="not found"), 404
    BOOKS.pop(i); return "", 204
```

CHẠY THỬ BẰNG CURL

```
# PATCH — đổi 1 field
$ curl -X PATCH \
localhost:5000/books/1 \
-H "Content-Type: application/json" \
-d '{"price":19.99}'
# PUT — thay sạch
$ curl -X PUT \
localhost:5000/books/1 \
-d '{"title":"New","author":"X"}'
# DELETE — trả 204
$ curl -X DELETE -i \
localhost:5000/books/1
```

LỖI SINH VIÊN HAY MẮC

1. Trộn PATCH và PUT — PATCH *không* xóa field không gửi.
2. Cho client đổi id — không bao giờ.
3. PUT 200 thay, 201 nếu *tạo mới*.

Nâng cấp **GET /books** thành production-ready

Bốn phần: **pagination**, **filtering**, **HATEOAS links**, **Cache-Control** — tất cả trong một endpoint.

```
# app.py — GET /books nâng cấp với pagination + filter + HATEOAS
from flask import Flask, jsonify, request, make_response
app = Flask(__name__)
# —— tham số phân trang
DEFAULT_SIZE, MAX_SIZE = 20, 100
# —— list + filter + paginate + links
@app.get("/books")
def list_books():
    try:
        page = int(request.args.get("page", 1))
        size = int(request.args.get("size", DEFAULT_SIZE))
    except ValueError:
        return jsonify(error="page and size must be int"), 400
    page = max(page, 1); size = max(min(size, MAX_SIZE), 1)
    # filter: author chính xác, q tìm trong title
    flt = BOOKS
    a = request.args.get("author")
    if a: flt = [b for b in flt if b["author"].lower()==a.lower()]
    q = (request.args.get("q")or "").lower()
    if q: flt = [b for b in flt if q in b["title"].lower()]
    # paginate
    total = len(flt); start=(page-1)*size; end=start+size
    items = flt[start:end]; last=(total+size-1)//size
    # HATEOAS links
    def u(p): return f"/books?page={p}&size={size}"
    links = {"self":{"href":u(page)},
            "first":{"href":u(1)},
            "last":{"href":u(max(last,1))}}
    if page > 1: links["prev"]={"href":u(page-1)}
    if end < total: links["next"]={"href":u(page+1)}
    body = {"data":items,
            "pagination":{"page":page,"size":size,"total":total,"total_pages":last},
            "_links":links}
    resp = make_response(jsonify(body), 200)
    resp.headers["Cache-Control"]="public, max-age=30"
    return resp
```

CÁCH GỌI

```
$ curl /books?page=2&size=10
```

```
$ curl /books?author=Orwell
```

```
$ curl /books?q=clean
```

```
$ curl -H "Accept: application/json" \
```

```
/books
```

PHÂN TRANG MẶC ĐỊNH

page=1, size=20 nếu không có query. *size>100* bị cap về 100.

Cache key chỉ phụ thuộc URL — filter khác nhau sẽ cache riêng.

RESPONSE MẪU

```
{
  "data": [
    {"id":11,"title":"Clean Code"},
    {"id":12,"title":"Clean Arch"}
  ],
  "pagination": {
    "page":1, "size":20,
    "total":2, "total_pages":1
  },
  "_links": {
    "self": {"href":"/books?page=1"},
    "first": {...},
    "last": {...}
  }
}
```

Tổng kết

01

REST là kiểu kiến trúc,
không phải JSON qua HTTP

02

Status code mang nghĩa,
body mang dữ liệu

03

Idempotency là hợp đồng
với client

04

Cache dựa trên URL và
Authorization

05

PUT thay, PATCH sửa,
POST tạo

BÀI TẬP VỀ NHÀ

BÀI TẬP 1

Refactor /orders sang SQLite

Thay BOOKS list thành SQLite (sqlite3, không cần ORM). Giữ nguyên URL contract. Bổ sung [GET /orders/<oid>](#).

Deadline: trước Buổi 3 · nộp app.py + dump database.

BÀI TẬP 2

Audit một public API thực

Chọn 1 API public (GitHub, Stripe, Shopify...). Liệt kê 5 endpoint — cho biết method, status code, headers, có RESTful không.

Nộp: file markdown 1-2 trang.

BÀI TẬP 3

Conditional requests với ETag

Cho [GET /books/<id>](#) trả kèm ETag; hỗ trợ [If-None-Match](#) trả 304 khi không đổi.

Gợi ý: hash theo nội dung, lưu cùng record.

Bốn cấp độ — từ “chỉ dùng HTTP” đến REST

CẤP CAO HƠN = GẦN REST HƠN

3

Hypermedia Controls

HATEOAS — links trong response

2

HTTP + Resources + Verbs

URLs định danh resource; dùng GET/POST/PUT/DELETE

1

Resources

Mỗi resource một URL; một endpoint HTTP duy nhất xử lý mọi thứ

0

The Swamp of POX

Một URL duy nhất, một HTTP method (thường POST), XML đầu đó: SOAP và RPC cũ

PHÊ BÌNH CỦA FIELDING (2010)

Mô hình này *không chính xác*, REST là tập ràng buộc rời rạc, không phải cấp bậc. Một API có thể thỏa mọi ràng buộc trừ HATEOAS vẫn tốt hơn một API chỉ dùng hypermedia sai cách.

CÁCH DÙNG HỮU ÍCH

Dùng làm **checklist giao tiếp** với stakeholder: API hiện tại ở level mấy? Level 2 (HTTP + resources + verbs) là mục tiêu thực tế của 90% API production.